Safe and Programmable OS-Level GPU Resource Management with eBPF

Anonymous Author(s)

Abstract

GPU resource management policies govern how a system places memory and schedules GPU tasks. They play an important role in determining application performance, and the ideal policy for a given deployment often depends on low-level details such as the application, workloads, and hardware. AI agents would make it possible to automatically optimize GPU resource management policies, but require three properties of a system: safety, to ensure that a broken policy cannot cause a system crash; dynamism, to enable policy changes without restarts; and full-stack, to ensure that the policy can observe device and application states and control low-level hardware. Unfortunately, existing techniques for GPU resource management fail to meet these needs.

We introduce gpubpf, a new system that provides an OS interface for GPU resource management that is safe, dynamic, and full-stack by turning to eBPF: gpubpf policies include eBPF callback functions that the system invokes on a specified event (e.g., when code reaches a function). The system includes a novel execution model that separates the effect of a callback from its triggering event. Namely, gpubpf supports policies that asynchronously transition resources between states, potentially invoking multiple transitions at a single point-in-time. We implemented gpubpf on the Linux NVIDIA GPU driver and find that AI-generated policies improve throughput by 1.76× over application-tuned baselines and reduce tail latency by up to 2×, without application modifications. Moreover, we find that there were zero OS kernel panics through the optimization process, highlighting gpubpf's safety.

1 Introduction

GPU resource management policies critically influence application performance. These policies include memory placement, migration, and GPU task scheduling. Recent research shows their substantial impact on Unified Virtual Memory (UVM) placement [3, 22, 27, 30] and GPU scheduling strategies [14, 38, 47]. For instance, optimal eviction and prefetch policies can improve performance by up to 73% under memory oversubscription [15, 22, 25].

Unfortunately, finding an optimal GPU resource management policy is challenging. The optimal memory and scheduling policies vary significantly across workloads, hardware, and deployment contexts [1, 5, 26, 28, 35, 36, 40, 46], with no one policy dominating the others. For example, aggressive memory prefetching improves latency for LLM MoE inference by reducing memory stalls [15, 22], but increases the latency of graph analytics queries because it over-saturates memory bandwidth [15]. Additionally, throughput-oriented scheduling improves average latency of vector search but degrades tail latency for interactive inference [1, 14].

We argue that the community should employ AI agents to automatically optimize programmable GPU resource management policies. AI-driven exploration can handle the vast, workload-specific policy space that makes manual tuning challenging. Agents can now generate complex code and optimize policies in domains such as compiler heuristics [17], datacenter scheduling, LLM serving [8, 20], training strategies [12], and CPU scheduling [13, 52]. In these use-cases, agents are generating fully fledged policy programs rather than simply tuning configuration parameters.

We derive three properties that systems should provide to enable AI agents to optimize programmable GPU resource management policies effectively without sacrificing system health. Namely, a system should support programmable policy interfaces that are safe, dynamic, and full-stack. A policy interface is safe if it ensures that new policies cannot cause the system to crash; dynamic if it supports policy updates without requiring the system and applications to be restarted; and full-stack if it supports cross-application and device-internal visibility, as well as low-level hardware control.

Existing programmable GPU resource management techniques fail to simultaneously meet these requirements. User-space frameworks [7, 16, 31, 38] are safe and dynamic but are not full-stack since policies lack cross-application visibility and hardware control. Other approaches modify the OS kernel driver [14, 23, 24, 47, 51], which fails to meet all three of the requirements.

In this paper, we propose gpubpf, a new system that provides an OS interface for GPU resource management policy that is safe, dynamic, and full-stack. Similar to prior work on resource management [4, 29, 54], gpubpf supports eBPF-based policies. Users write callback functions that manage system resources and are called on user-defined events (e.g., when the system reaches a specific code location, or periodically). gpubpf's use of eBPF ensures safety, since the system uses an eBPF-style verifier before running new policies, and

dynamism, since it can replace policies without service interruptions. To provide a full-stack interface, gpubpf supports eBPF callbacks that execute within the OS kernel driver and GPU device.

gpubpf models GPU resource management as transitions on resource state machines (§3.2). State machines define valid states and transitions: GPU memory chunks transition between host memory and device memory through eviction and prefetching; compute contexts transition through states such as running, ready, and finished via scheduling and preemption operations. eBPF policies specify the transitions that the system should take on its GPU resources.

Alas, applying the CPU eBPF programming model to GPU resource management is not straightforward. The GPU is a discrete processor with its own execution model and resources, and policy decisions made in the host take effect on a separate device. In particular, we identify three challenges: first, cross-device operations like migrating data or dispatching commands take milliseconds, too slow for synchronous callbacks and too costly for purely reactive decisions. Second, unlike independent subsystems on the CPU, limited device memory and bandwidth further couple memory and scheduling, as decisions about data placement directly affect which workloads can execute. Third, GPU applications employ OS kernel bypass, which requires gpubpf to support eBPF callbacks from events in user-space applications. Traditional eBPF does not allow such callbacks to modify OS state required to implement resource management policies because of safety concerns.

gpubpf addresses these challenges through a novel execution model for eBPF-based resource management that separates the effect of a callback from its triggering event. Unlike prior eBPF-based systems (e.g., sched_ext [29], cache_ext [54], and XDP [4]), where each callback makes a single synchronous decision at a fixed event, gpubpf allows callbacks that asynchronously transition resources at user-defined events. In particular, gpubpf callbacks invoke transitions when a given event occurs, and the system asynchronously transitions the resource between states. gpubpf callbacks can initiate multiple state transitions within a single event. Finally, gpubpf supports user-defined events that can execute policy code from anywhere in the GPU stack (application, OS kernel driver, or GPU device). Critically, to enable safety with user-defined events, gpubpf policies only initiate predefined transitions; they cannot create invalid transitions that could corrupt resources.

We implement gpubpf on Linux by extending the NVIDIA open GPU OS kernel modules [32] and providing an eBPF SIMT verifier and JIT runtime for GPU devices. We evaluate gpubpf across LLM inference, GNN training, vector search, and multi-tenant scenarios. gpubpf-based policies improve throughput by up to 4.8× over framework offloading and 1.76× over application-tuned baselines, and reduce tail latency by up to 2×, with low overhead and no application modifications or restarts. We show that gpubpf enforces safety by using AI agents to generate all of the 59 evaluated eBPF policies, including eviction ordering, prefetch strategies, scheduling logic, and device-side work-stealing. gpubpf ensured that there were zero OS kernel panics throughout this process. In summary, our contributions are:

- an execution model for eBPF-based resource management that separates the effect of a callback from its triggering event.
- gpubpf, a system that provides a programmable GPU resource management policy interface that is safe, dynamic, and full-stack.
- an evaluation of gpubpf across four GPU workloads, achieving up to 4.8× throughput improvement and 2× tail latency reduction, with all 59 policies generated by AI agents and no OS kernel panics.

2 Background and Motivation

2.1 GPU Systems and Workload Diversity

As shown in Figure 1, we focus on three GPU resource management domains across the software stack: memory placement and migration, compute scheduling, and on-device thread scheduling. User-space frameworks (e.g., vLLM [26], Salus [50], PILOT [37]) leverage application semantics (e.g., neural structures, latency constraints) to implement request scheduling and KV-cache offloading policies. Kernel drivers manage memory, scheduling, and multi-tenant isolation via privileged hardware mechanisms (MMU, interrupts), typically using fixed, application-agnostic algorithms (e.g., LRU eviction) under Unified Virtual Memory (UVM), which transparently handles oversubscription through host-device page migrations. GPU devices execute kernels via SIMT (Single Instruction, Multiple Threads); on NVIDIA hardware, 32-thread warps run instructions in lockstep on SMs, with internal states (e.g., warp access patterns) opaque to host drivers.

GPU workloads exhibit diverse resource access patterns. Results from gpubpf's eBPF-based tracing tools (§5) confirm this diversity: Figure 2 captures memory page-fault addresses over time under UVM, showing that LLM MoE inference exhibits periodic stride bursts from expert activations; GNN training shows irregular random access from pointer-chasing graph traversal; and vector search alternates between sequential scans (index build) and random probes (query) [5, 35]. Figure 3 reveals a 127× SM scheduling load imbalance in a vector-add GPU kernel, invisible to host-side drivers.

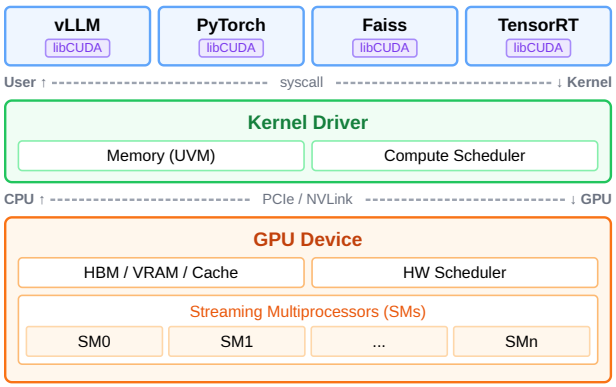

Figure 1. GPU system stack: user-space frameworks, kernel driver (memory and scheduling), and GPU hardware.

These diverse patterns require workload-specific policies across all three domains. UVM eviction and prefetch policies alone can impact execution time by up to 73% [15, 22]: MoE inference favors specialized eviction over default LRU [26, 39], while GNN training demands aggressive sequential prefetch [28, 46], but each can degrade other workloads. Compute scheduling requires adaptive priority and preemption policies as multi-tenant demands fluctuate [14, 47], and SM load imbalance benefits from custom thread scheduling. Yet current drivers and firmware enforce a single fixed policy.

2.2 Agent-Based Policy Exploration

Agentic OS policy exploration fits the large GPU policy space but requires programmable interfaces missing in current GPU stacks. Unlike parameter-tuning methods, AI agents [13, 52] directly generate and iteratively refine policy logic to explore richer, workload-specific strategies. However, existing GPU stacks provide only fixed knobs (e.g., `cudaMemAdvise`, driver module parameters), lacking programmability needed for complex heuristics such as workload-aware eviction or adaptive prefetching.

Effective GPU policy optimization requires an OS interface with three key properties: (1) *safe*, robustly handling generated policies without kernel panics or GPU corruption; (2) *dynamic*, rapidly deploying and iterating policies at runtime without application restarts or kernel rebuilds; and (3) *full-stack*, observing GPU-internal states, coordinating across tenants, and controlling driver-level decisions like eviction and fault-path prefetching.

2.3 Limitations of Existing Approaches

Existing approaches fail to simultaneously provide a safe, dynamic, and full-stack programmable OS interface for GPU resource management.

Driver-Level Policies. Driver-level approaches either expose fixed knobs (e.g., `cudaMemAdvise`, driver module parameters) that select among preset behaviors, or embed custom logic directly into driver code. Systems like TimeGraph [23], Gdev [24], and OS kernel interception methods [51] embed scheduling and memory-management logic into driver modules. More recent work such as GPREEMPT [14] and GCAPS [47] add priority-based preemption, and LithOS [11] re-implements runtime stacks with OS-like abstractions. Despite improved performance, all these approaches require OS kernel or driver modifications to change policy: they are neither safe (bugs can crash the OS kernel) nor dynamic (policy changes require recompilation and restarts), and still lack GPU-internal observability for full-stack control.

Host User-Space Runtimes and Libraries. User-space frameworks [7, 11, 16, 31, 38] offer programmability but face three limitations. First, policies are application-bound, requiring per-framework implementation and preventing reuse across ecosystems. Second, process-level isolation prevents cross-tenant visibility and coordinated memory-scheduling decisions across tenants [48, 50]; hardware partitioning such as MIG imposes fixed resource boundaries that cannot be dynamically rebalanced. Third, user-space code cannot access privileged driver mechanisms, leaving policies constrained to GPU kernel launch boundaries and unable to execute on latency-critical paths (e.g., page-fault handlers), perform precise preemption, or exploit GPU-side warp-level access patterns. In summary, these frameworks are safe and dynamic but not full-stack.

3 gpubpf Design

This section presents the design of gpubpf, a programmable OS subsystem for safe, dynamic, full-stack GPU resource management. We describe the system workflow (§3.1), resource state machines (§3.2), GPU-specific eBPF challenges (§3.3), and our asynchronous execution model (§3.4).

3.1 gpubpf Workflow

gpubpf extends eBPF (extended Berkeley Packet Filter) to GPU resource management. eBPF programs are callbacks attached to hooks such as `struct_ops`, `kprobe`, and `uprobe`, enabling safe execution within the OS kernel via load-time verification and JIT compilation. The verifier ensures termination, memory safety, type correctness, and restricts calls to approved helpers and kfuncs (OS kernel functions). As

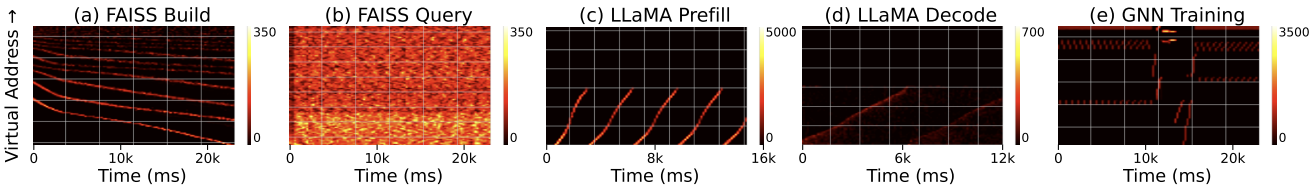

Figure 2. Page-fault address traces over time across four workloads. Each pattern implies a different optimal prefetch/eviction strategy; no single policy fits all.

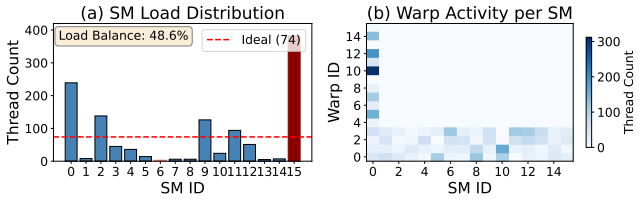

Figure 3. GPU thread scheduling imbalance observed via eBPF tracing. (a) SM load distribution: 127× imbalance. (b) Warp activity heatmap across SMs.

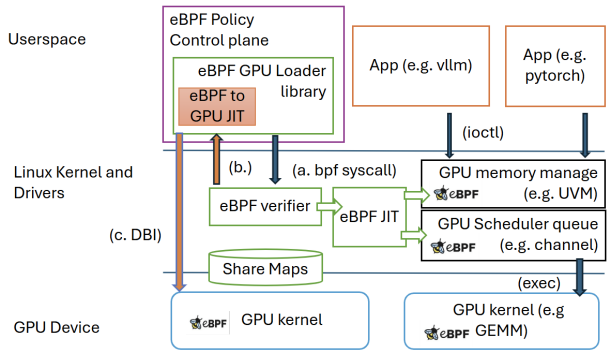

Figure 4. gpubpf architecture. (a) eBPF syscall deploys policies to verifier and driver hooks; (b) GPU JIT after verification; (c) DBI injects trampolines into GPU kernels.

shown in Figure 4, gpubpf extends this pipeline to GPUs: policies build and load with standard eBPF tools, contain both host and device components, run across the host driver and GPU, are verified with SIMT-aware checks (§3.5), JIT-compiled to CPU or GPU code (e.g., PTX), use cross-layer maps, and are dynamically installed via OS kernel hooks and DBI without application or driver restarts.

3.2 Resource State Machines

gpubpf defines three resource domains. Memory chunks move between host and device memory via eviction and prefetching. GPU kernels (launched tasks) and GPU thread-blocks (parallel execution units within kernels) transition among ready, running, and finished states through scheduling and preemption operations.

3.3 GPU-Specific Challenges for eBPF Extensibility

CPU eBPF does not directly transfer to GPUs. CPU eBPF callbacks make synchronous decisions at fixed transition points, immediately affecting local resources, and subsystems are extended independently. In contrast, GPUs are discrete processors: host policy decisions affect resources asynchronously on a separate device. Thus, three architectural mismatches introduce key challenges:

Cross-device latency. CPU eBPF callbacks execute synchronously, but GPU operations (e.g., page migration across PCIe) take milliseconds (§2.1). GPU policies must proactively initiate asynchronous actions instead of reacting only at faults.

Memory-scheduling coupling. CPU extensibility treats scheduling and memory independently, but GPU page faults simultaneously trigger memory events and scheduling stalls due to limited device memory. GPU policies must jointly coordinate these domains.

OS kernel bypass. GPU runtimes bypass the OS kernel (e.g., user-space command buffers, doorbells), revealing only coarse-grained resource events to drivers, not fine-grained GPU kernel launches or application phases. GPU policies must attach to user-space application functions, yet standard eBPF prohibits these callbacks from modifying OS state required for resource management. Exposing GPU driver controls without addressing these mismatches risks resource leaks, hardware hangs, or OS kernel panics.

3.4 Asynchronous Execution Model

To address these challenges, gpubpf separates callback triggers from their effects (Figure 5). Policies, as verified eBPF programs invoking OS kernel functions (*kfuncs*), can attach to driver events, user-space uprobes, and device-side events. Each trigger asynchronously initiates multiple proactive

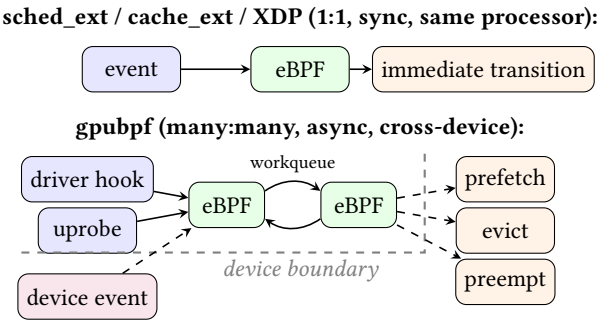

Figure 5. Policy execution model: cpu policies vs gpubpf.

or deferred state transitions across memory and scheduling domains without blocking, coordinated through eBPF workqueues. For example, a uprobe on `cudaLaunchKernel` concurrently initiates a 6.6 ms DMA prefetch based on device observations and preempts another tenant to free GPU memory.

To ensure OS kernel safety, verified kfuncs restrict policy actions to two dimensions: *selection* (choosing resources, e.g., eviction order, tenant preemption priority) and *timing* (when to act, e.g., proactive prefetch). The driver always controls underlying state machines; policies only select ordering and timing of driver-managed operations, cannot allocate memory or remove tasks, and revert to FIFO defaults under resource pressure or misconfiguration. Currently, a single policy program is loaded system-wide but may internally differentiate tenant treatment for priorities, similar to sched_ext; per-tenant isolation is future work.

3.5 Runtime Verification and Optimizations

gpubpf's threat model mirrors standard eBPF: administrators are trusted, but policy code may be buggy or misconfigured. The trusted computing base (TCB) includes the OS kernel, gpubpf driver module, GPU compiler backend, and GPU driver/firmware. Host-side verification directly reuses the Linux eBPF verifier (§4).

3.5.1 SIMT-aware Verification. Because GPUs execute in SIMT (Single Instruction, Multiple Threads) mode, device-side eBPF programs must maintain warp-level execution semantics within eBPF's scalar model. gpubpf enforces warp-uniformity via SIMT-aware verification: it distinguishes *warp-uniform* (identical across warp threads, e.g., region IDs) from *lane-varying* (thread-local state) values. Lane-varying values cannot affect control flow or external side effects. Branch conditions, loop bounds, and map-update keys must be warp-uniform or warp-aggregated. Additionally, gpubpf forbids GPU-wide barriers, global synchronization, non-uniform atomics, and it bounds resource usage per policy hook.

3.5.2 Warp-level Execution Optimizations. Even after SIMT verification, scalar per-thread eBPF execution incurs overhead (e.g., duplicate map updates). To resolve this, gpubpf introduces warp-level aggregated execution: lanes compute local results without affecting control flow or shared state; these results are aggregated warp-wide, and a warp leader executes the policy once, broadcasting decisions back (§4).

3.5.3 Hierarchical Cross-layer Maps for State Coordination. To share state across CPU and GPU contexts, gpubpf introduces cross-layer maps: logically unified key-value stores accessible by host driver hooks, GPU-side policies, and user-space control planes, verified through standard eBPF. Physically, maps are hierarchical shards dynamically placed by the runtime based on access patterns, latency, and capacity constraints (§4). Following eBPF's philosophy, gpubpf maps use eventual consistency: periodically merging device-local shards into canonical snapshots at GPU kernel completions. This model suffices as policies rely on approximate statistics; occasional staleness may affect optimality but cannot violate correctness invariants like memory integrity.

3.6 Example: MoE Expert Offloading

We illustrate this model using Mixture-of-Experts (MoE) offloading, where models exceed GPU memory capacity. Since all experts do not fit and activate concurrently, the GPU driver continuously evicts and migrates pages. The default driver's LRU eviction and prefetching are unaware of expert activations, causing costly PCIe DMA stalls.

A gpubpf policy uses three cooperating eBPF programs at different layers (Figure 6), coordinating via a shared cross-layer map. First, during GPU kernel execution, a device-side eBPF handler inserted at warp entry tracks warp-level accesses to expert pages, recording per-page access counts for each warp in a hierarchical cross-layer map. Second, a uprobe eBPF program attached to `cudaLaunchKernel` proactively reads these counts to predict upcoming expert activations and asynchronously initiates cross-device prefetches (`bpf_gpu_prefetch_async`), overlapping the millisecond-scale memory migration with computation to avoid expensive stalls (*timing*). If memory pressure arises, this same program coordinates across memory and scheduling by preempting lower-priority GPU tasks via `gdrv_sched_preempt` (*selection*). Finally, upon memory eviction (*evict_prepare*), a driver-side eBPF handler leverages the same expert access counts to evict cold pages first (LFU ordering), protecting hot experts (*selection*). Unlike framework-level offloading that often migrates experts as atomic units, gpubpf operates

```
SEC("gpu_dev/warp_entry")
int moe_observe(gdev_page_access_ctx_t *ctx){
  u64 va = ctx->va & EXPERT_MASK;
  u64 *freq = bpf_map_lookup_elem(
                  &expert_freq, &va);
  if (freq) fetch_and_add(freq, 1);
  return 0;
}
SEC("uprobe/cudaLaunchKernel")
int moe_prefetch(struct pt_regs *ctx) {
  u64 next = predict_next_expert(ctx);
  bpf_gpu_prefetch_async(
                  next, EXPERT_SIZE);
  if (bpf_gpu_mem_pressure() > THRESHOLD)
    gdrv_sched_preempt(BE_CONTEXT);
  return 0;
}
SEC("gpu_mem/evict_prepare")
int moe_evict(gdrv_mem_remove_ctx_t *ctx){
  gmem_region_t *r = ctx->region;
  u64 *freq = bpf_map_lookup_elem(
                  &expert_freq, &r->va);
  if (freq && *freq > HOT_THRESHOLD)
    bpf_gpu_move_tail(&ctx->evict_list, r);
  else
    bpf_gpu_move_head(&ctx->evict_list, r);
  return 0;
}
```

Figure 6. MoE expert offloading policy

at page granularity, enabling finer-grained compute-transfer overlap. Each eBPF program captures partial state at different layers; cross-layer maps unify this fragmented visibility to orchestrate state transitions across host and GPU.

4 Implementation

We implemented gpubpf on Linux 6.15, extending the OS kernel with a standalone file (~1 KLOC) that handles hook registration and verifier extensions. We integrated lightweight instrumentation (~100 LOC) into NVIDIA's open GPU OS kernel modules to expose memory and scheduling state transitions as GPU-specific kfuncs (§4.1). Our user-space loader and JIT infrastructure (~10 KLOC) builds upon bpftime's framework [53] for device-side code generation, while an LLVM-based backend (~1 KLOC) translates the restricted gpubpf eBPF instruction subset into GPU device code (PTX) (§4.2). Although our implementation targets NVIDIA GPUs, the host-side design aligns with generic Linux abstractions (HMM/`migrate_vma` [2, 41], DRM scheduler [42, 43]), and the device-side can be extended to generate vendor-neutral IR [10] for future portability.

4.1 Host-side Integration

Memory State-Transition Hooks. gpubpf extends the NVIDIA Open GPU OS Kernel Modules [32], instrumenting the UVM module at four code locations: *activate* (chunk allocation), *access* (page fault), *evict_prepare* (eviction candidate selection), and *prefetch* (proactive migration). At each hook, the driver constructs a context (e.g., 2MB-aligned region range, fault address, block size) and invokes the attached eBPF handler. Eviction hooks operate at 2MB granularity; prefetch hooks operate at page granularity and can prefetch as many pages as needed. We extend eBPF to allow each handler to initiate asynchronous effects via kfuncs such as `bpf_gpu_prefetch_async` for cross-device DMA or `gdrv_sched_preempt` for scheduling preemption with `bpf_wq` (workqueue on background threads). The driver maintains a doubly-linked eviction list; handlers may reorder regions via provided helpers, but the driver retains eviction authority under memory pressure.

Scheduling Hooks. We instrument Time-Slice Group (TSG) code locations, including initialization (`task_init`) and teardown (`task_destroy`). The eBPF policy can configure scheduling parameters such as TSG priority, timeslice duration, and runlist interleave frequency via the `bpf_gpu_set_attr` kfunc. Policies triggered from uprobes or tracepoints can also preempt running GPU kernels precisely with preemption-request kfuncs.

4.2 GPU Runtime

Verification and JIT Compilation. We reuse Linux's eBPF verifier for standard host and device safety checks. After verification, our loader forwards device-side bytecode and BTF metadata to a user-space backend for GPU PTX generation. Additional SIMT-aware passes (§3.5) perform forward dataflow analysis over the CFG, propagating per-register uniformity tags from BTF annotations to reject warp-uniformity violations. During JIT compilation, we inline helper functions and map accesses. Unlike the workshop version [49], which relies on atomic synchronization, gpubpf avoids cross-SM primitives to prevent hardware stalls.

Device-Side State-Transition Hooks. gpubpf hooks device-side state transitions (GPU kernel launch, memory access, execution-phase boundaries) by using ptrace to pause applications, intercepting CUDA runtime APIs, extracting GPU kernel PTX, inserting binary trampolines, and reloading instrumented GPU kernels without application recompilation or restart. Trampolines insert prologue/epilogue logic for SIMT register preservation. In each trampoline, per-lane inputs are aggregated using `__ballot_sync` and `__shfl_sync`, with a warp leader

selected via `__ffs(__activemask())` to execute the scalar eBPF handler once per warp, broadcasting results via shuffle instructions. For block-level scheduling, gpubpf implements policies using persistent GPU kernels that poll and claim logical work units; on Blackwell GPUs, we use cluster launch control APIs [34] to schedule thread blocks. For memory observation, gpubpf attaches eBPF handlers at memory instructions, GPU kernel function boundaries, and tracepoints to capture access patterns and issue policy hints to the host driver via cross-layer maps (e.g., device L2 prefetch can trigger a host callback for extended prefetch).

Hierarchical Maps and Cross-Domain State. We realize cross-layer maps (§3.5.3) by placing shards in GPU-accessible memory tiers: cold state in host DRAM, hot state in GPU global memory, and per-warp data in GPU shared memory. A runtime daemon asynchronously flushes GPU-local shards to host-visible canonical instances, providing coherent snapshots without synchronization overhead. Shard updates occur at warp granularity through warp-leader threads without GPU atomic operations: handlers accumulate counters in warp-local registers, aggregate per warp, and store into GPU-local shards for periodic host flush.

5 Evaluation

Our evaluation addresses the following research questions:

- **RQ1 (Single-Tenant Management):** How much performance gain can gpubpf's programmable memory and scheduling policies provide on oversubscribed single-tenant workloads?
- **RQ2 (Agent-Driven Exploration):** Does gpubpf's eBPF containment model make GPU policy space exploration safe and productive for AI agents?
- **RQ3 (Multi-Tenant Management):** Does gpubpf improve tail latency, throughput, and resource fairness compared to user-space and global policies in multi-tenant settings?
- **RQ4 (Overhead):** What is the overhead of gpubpf's core mechanisms and observability capabilities?

5.1 Methodology and Setup

We evaluate gpubpf on two machines: Server A with Intel Core Ultra 9 285K (24 cores), 128 GB DDR5 RAM, and NVIDIA RTX 5090 GPU; Server B with dual Intel Gold 6138 (80 cores), 256 GB RAM, and NVIDIA P40 GPU. Unless otherwise specified, experiments run on Server A, averaging 10 trials using geometric mean. We compare gpubpf against three classes of baselines: (1) driver defaults (UVM LRU/FIFO eviction, native GPU scheduler), (2) application-level hints (`cudaMemAdvise`), and (3) framework-managed policies (e.g., llama.cpp `ncmoe`, vLLM CPU offload, LMCache) where applicable. We use PyTorch (version 2.9.0) and vLLM (version 0.11.0) with custom allocators, llama.cpp (version 7101), and Faiss (version 1.13.0) with UVM build config. All tests with gpubpf policies require no application modifications.

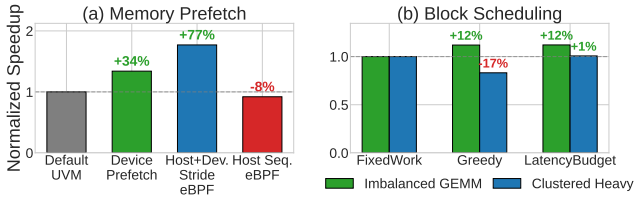

Figure 7. Single-tenant policy evaluation (normalized speedup, higher is better): (a) memory prefetch policies on vector-add; (b) block scheduling policies on GEMM.

5.2 RQ1: Single-Tenant Memory and Scheduling Policies on GPU kernels

Memory Prefetch Policies. We evaluate host-device prefetch coordination using a vector-add GPU kernel [33] with stride access pattern and 40GB working set (1.25× oversubscription). Device-side L2 prefetch instructions (`prefetch.global.L2`) trigger non-blocking page faults, while host-side callbacks extend prefetching to additional pages. As shown in Figure 7a, device-only prefetch (45 LOC) at thread entry achieves 1.34× speedup while combined host-device stride-based eBPF prefetch (472 LOC host + 45 LOC device) achieves 1.77×. However, sequential prefetch (375 LOC) degrades performance by 8% due to pattern mismatch.

Single-Tenant Block Scheduler. We apply a gpubpf-based thread block scheduler to workloads with load imbalance. Using cluster-style launch where persistent worker blocks pull work units via `should_try_steal`, we evaluate three policies: FixedWork (static block assignment, no scheduler), Greedy (always-steal, 16 LOC), and LatencyBudget (workload-aware with per-block time limits, 19 LOC). Figure 7b compares these policies across two GEMM workloads. Under moderate imbalance, both Greedy and LatencyBudget reduce latency by ~11%. Under heavy-tail workloads where 10% of blocks perform 100–200× more work, Greedy increases latency by 20% due to contention on heavy blocks. LatencyBudget, which caps per-block stealing time, matches fixed-work baseline. This highlights gpubpf's programmability: different workloads require different scheduling strategies.

5.3 RQ2: Agent-Driven Policy Exploration Case Studies

We provided an AI coding agent (Claude Code with Opus 4.5) with gpubpf's eBPF interfaces; the agent autonomously generated all policy logic without human review or guidance, while humans authored only the initial framework and benchmark harness. All 59 policies evaluated were produced via a fully automated generate-test-iterate loop. Over 20 days, the agent performed 974 benchmark runs and 244 code edits, exploring 59 distinct policies and consuming 5.5M tokens ($388 API cost); approximately 75% of explored policies yielded negative results before being iteratively corrected. We documented 50 safety events (24 logic bugs, 18 performance regressions, 2 verifier rejections, 2 GPU verifier-caught overflows, 4 others), all recovered without OS panics or data corruption, typically within minutes and without application restarts. Equivalent bugs in modified GPU drivers risk kernel panics, requiring recompilation and service interruptions rather than instant policy detachment. We conclude with four case studies showing performance gains and the exploration process.

***Expert Offloading (llama.cpp GPT-OSS-120B).*.** We evaluate gpubpf on GPT-OSS-120B MXFP4 MoE (59 GB) in llama.cpp on RTX 5090 (32 GB), requiring 1.84× oversubscription. We compare five configurations: framework-managed CPU offloading (`ncmoe=32/64`), default UVM, UVM with application-level hints (`cudaMemAdvise`), and gpubpf with eBPF-based host-device prefetching (§3.2). Figure 8 shows prefill and decode throughput. For decode (memory-bound), gpubpf achieves **1.76× speedup** over application-tuned UVM hints (`cudaMemAdvise`) and 4.8× over framework offloading (`ncmoe=64`). These hints improve decode over default UVM but require application modification and degrade prefill by 40%; gpubpf outperforms hints on decode while preserving prefill within 4% of default UVM, without application changes. For prefill (compute-bound), framework offloading (`ncmoe=32`) achieves 13% higher prefill throughput than gpubpf, but decode dominates end-to-end latency, so the decode improvement outweighs the prefill gap.

The agent arrived at the final policy through iterative exploration: sequential prefetch degraded decode throughput due to mismatch with MoE stride patterns (consistent with the 8% degradation in Figure 7a), leading to stride-based prefetch aligned with regular weight layout within each expert layer. Device-side eBPF tracing of per-warp memory accesses revealed that expert activations follow predictable stride sequences and identified frequently-accessed shared layers. For eviction, the agent tested FIFO ordering before converging on LFU, which retains frequently-accessed shared layers while evicting cold expert pages. The final policy combines device-side access observation (45 LOC), host-side stride prefetch (472 LOC), and LFU eviction (304 LOC), totaling ~820 LOC (§3.2).

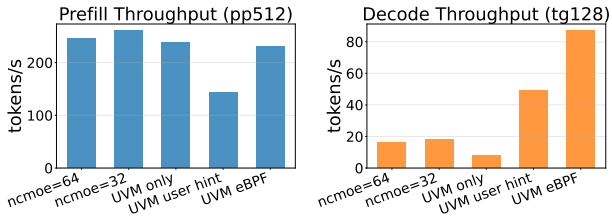

Figure 8. Prefill and decode throughput for GPT-OSS-120B MoE on llama.cpp.

***KV-cache Offloading (vLLM Qwen-30B MoE).*.** We evaluate gpubpf on the Qwen-30B FP8 MoE model (≈30GB) on RTX 5090 (32GB). The model nearly fills GPU VRAM, so the default no-offload configuration OOMs under load. We stress KV-cache growth by sending 100 concurrent requests from ShareGPT [45] (single-round, no prefix caching), resulting in 36–40GB total footprint (≈60K tokens KV-cache). Existing solutions offload KV-cache or experts but not both; UVM attempts on-demand migration of both, leading to thrashing. We compare vLLM's CPU-offload mode (`--cpu-offload-gb 8`) against gpubpf with UVM and a sequential prefetch policy. gpubpf improves mean and p99 time-to-first-token by 1.7–2× and decoding throughput by 1.3× over vLLM's default framework-managed offload (Figure 9), matching LMCache [9], a state-of-the-art KV-cache offloading framework. Notably, UVM without gpubpf policies performs worse than vLLM's offload, while gpubpf performs similarly to the best framework offloading, with better tail latency, dynamically and transparently.

Under memory pressure, the main challenge is managing two competing data types: KV-cache, exhibiting temporal locality (recent tokens accessed frequently during attention) and per-request spatial locality, and model weights with stride patterns in matrix operations. A uniform sequential prefetch initially caused mutual thrashing between KV-cache and weight pages. Using device-side tracing, the agent explored region-differentiated prefetch strategies, converging on adaptive aggressiveness guided by PCIe utilization and region type (stride for weights, sequential for KV-cache, 375 LOC), coupled with LFU eviction (304 LOC) retaining frequently accessed pages. The resulting policy (totaling ~680 LOC) effectively prevents thrashing seen with default LRU.

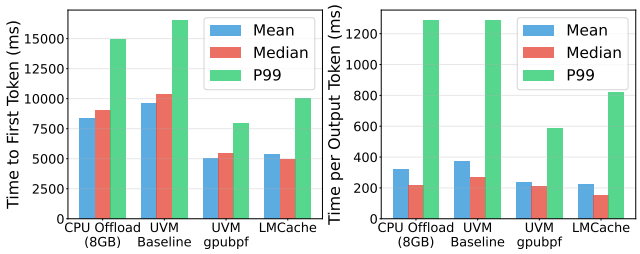

Figure 9. Time-to-first-token and decoding throughput on Qwen-30B MoE inference with vLLM.

Graph Neural Networks (GNN Training). We evaluate gpubpf on PyTorch-based GCN training with random graphs of varying sizes (1M–15M nodes, 10 edges/node). Figure 10 shows epoch time across five configurations: native GPU allocation (no UVM), UVM with and without user-space prefetching (`cudaMemPrefetchAsync`), and each UVM configuration with gpubpf eBPF-based optimization. With UVM enabled via a custom allocator, each epoch allocates memory on CPU and migrates to GPU on demand, incurring overhead. Developers can pre-migrate pages via `cudaMemPrefetchAsync`, achieving 5.5× speedup at moderate oversubscription, but this requires application modification. Without user hints, eBPF optimizes page fault handling via aggressive prefetching, achieving 2.65× speedup transparently. With both user-expressed prefetching and gpubpf, it achieves 1.44× additional speedup by prefetching larger regions and reducing blocking faults. Native GPU allocation (no UVM) achieves the highest performance when data fits in memory (1.89 s at 5M nodes) but fails with OOM beyond ~8M nodes. UVM with gpubpf prefetching enables training at 15M nodes (2.17× oversubscription) with near-native performance and no application modification.

gpubpf uses sequential prefetch (375 LOC) with a uprobe tracing PyTorch allocations, determining prefetch ranges via device-traced history. The agent tested deeper lookahead after single-block-ahead prefetch (K=1) improved epoch time by 21%, but found K=2 and K=3 degraded by 16%, and adaptive K (up to 6) by 46%. Suspecting lock contention initially, the agent identified PCIe bandwidth saturation as the true bottleneck, confirming one-block lookahead as optimal.

Faiss Vector Search. We evaluate gpubpf on vector similarity search using the SIFT dataset with an IVF4096,Flat index, scaling from 20M (9.5GB) to 50M (24GB) and 100M (48GB, exceeding GPU memory). IVF indexes have a two-level structure: frequently-accessed centroids and larger posting lists, and Faiss operations have two distinct phases: BUILD performs sequential scans while SEARCH issues random accesses. gpubpf reduces build time by 21–29% on

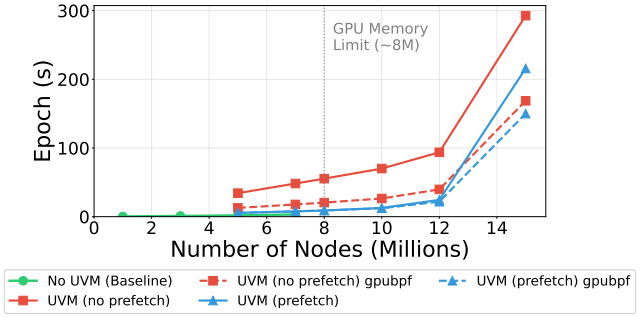

Figure 10. GCN training epoch time vs graph size on PyTorch.

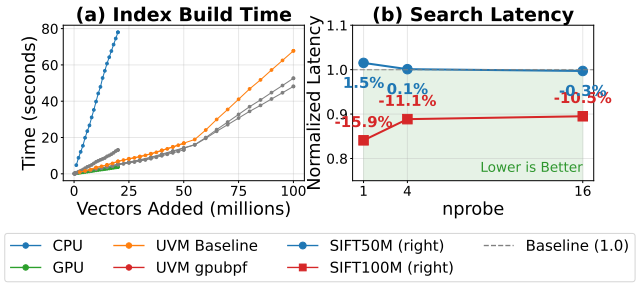

Figure 11. Faiss index build time and query throughput under oversubscription.

50M, scaling to 27–40% on 100M as memory pressure grows. For search workloads, gpubpf reduces latency by 10–16% across different `nprobe` settings despite the random access patterns inherent in ANN search. As shown in Figure 11, the converged policy (~890 LOC: sequential prefetch 375 LOC + stride prefetch 472 LOC + device-side 45 LOC) prefetches posting lists sequentially for oversubscribed datasets and uses stride prediction for K-means iterations during index construction; device-side eBPF programs also trigger prefetching of corresponding posting lists.

The agent tested 8 policies over 5 iterations. Initially, a momentum-based phase detector with cycle eviction improved BUILD but degraded SEARCH, as forward drift in search queries confused the detector. Fixing the classifier revealed eviction policy rather than phase detection controlled performance. Version three, pairing the corrected detector with default eviction, converged. A subsequent fast-path optimization introduced a state-machine bug (policy stuck in SEARCH mode), detected and fixed in one iteration. Only 2 of 8 policies yielded positive results.

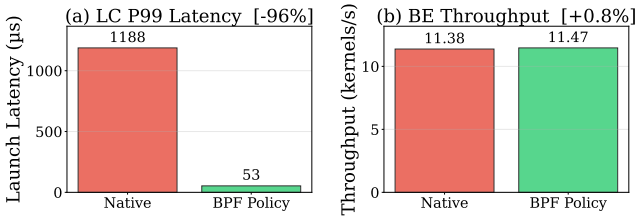

Figure 12. Multi-tenant GPU scheduling with gpubpf (LC: latency-critical, BE: best-effort).

5.4 RQ3: Multi-Tenant Memory, Bandwidth, and Scheduling

We evaluate gpubpf on multi-tenant GPU workloads, comparing per-tenant policies against global and framework-managed approaches.

Compute-bound Scheduler (LC + BE). We evaluate gpubpf's eBPF struct_ops scheduling policies on *compute-bound* (SM-limited) multi-tenant workloads, without memory oversubscription. 2 latency-critical (LC) processes and 4 best-effort (BE) processes run concurrently, each with 4 CUDA streams submitting 50 compute kernels. We compare the native scheduler against gpubpf with differentiated timeslices (LC: 1s, BE: 200μs) using a preemption policy equivalent to GPREEMPT's [14] priority-based timeslice scheduling, implemented as a gpubpf eBPF program (925 LOC) without unsafe modification to the driver source. Figure 12 shows that gpubpf reduces LC P99 launch latency by 96% (1188μs $\rightarrow$ 53μs). BE throughput remains unchanged.

Memory-bound Priority Differentiation. We evaluate gpubpf's memory policies for priority differentiation on *memory-bound* workloads under UVM oversubscription using HotSpot [6] (spatial locality), GEMM from PolyBench [18] (compute-intensive, memory-bound under UVM oversubscription), and K-Means [19] (sparse access). Each experiment runs two concurrent processes competing for GPU memory. Policy labels use priority values 0–100 (lower = higher priority): `Prefetch(lo,hi)` prefetches pages for processes in that range (454 LOC), while `Evict(lo,hi)` selects eviction victims accordingly (472 LOC). We compare against: (1) no-policy execution, (2) single-process runtime (Single 1×), and (3) theoretical optimum (2×Single 1×).

Figure 13 shows that without policies, both processes degrade severely due to memory thrashing. With gpubpf memory priority policies, total completion time improves by 55–92%, and high-priority processes complete 6–19% faster. Critically, for these *memory-bound* workloads, GPREEMPT-style [14] scheduler timeslice policies are *ineffective* (<1% improvement) because the bottleneck is UVM page faults and

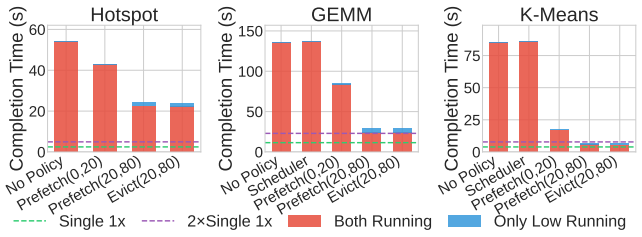

Figure 13. Multi-tenant memory priority differentiation. Single 1×: single-process time; 2×Single 1×: theoretical lower bound.

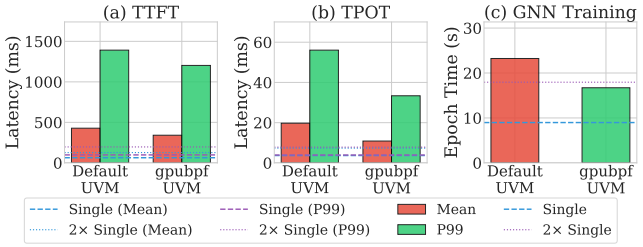

Figure 14. Two-tenant co-location: `llama.cpp` inference (LC) + GNN training (BE).

PCIe bandwidth, not GPU compute time. This contrasts with the compute-bound case above where the same GPREEMPT-equivalent scheduling policy is effective, showing that effective management requires selecting the right policy layer per workload.

Two-Tenant: High-Priority LLM Inference + Background Training. We run two tenants sharing a single GPU: T1 is a high-priority `llama.cpp` inference service (LC workload) serving 100 ShareGPT prompts at 0.2 RPS, and T2 is a background PyTorch GNN training job (BE workload) with 8M nodes requiring 36GB peak memory (oversubscribed). We compare default UVM (driver FIFO/LRU) against gpubpf per-tenant memory and scheduler policies (~926 LOC, combining Quota LRU, Tree-based Prefetch, and Dynamic Timeslice). Figure 14 shows mutual improvement: LC TPOT drops by 40–45% and TTFT by 14–20%, while GNN training simultaneously improves by 28%, approaching the ideal fair share baseline. This shows gpubpf reduces contention for both workloads rather than favoring one over the other.

5.5 RQ4: Overheads

We quantify gpubpf's mechanism and observability overhead.

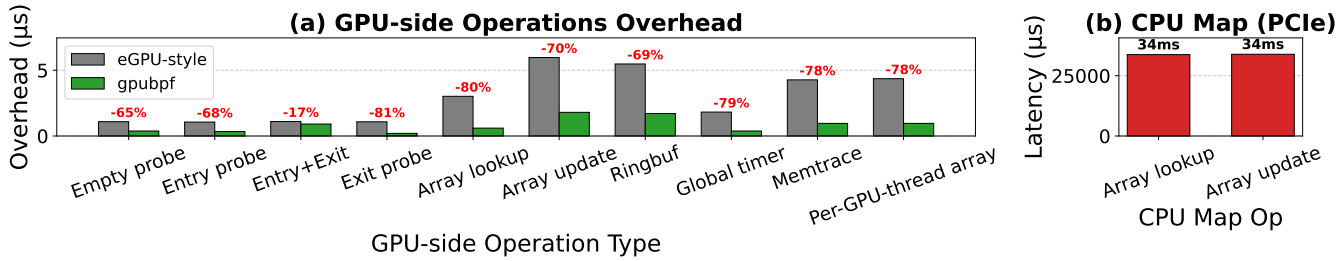

Figure 15. Device-side overhead. (a) Naive per-thread injection vs gpubpf's SIMT-aware execution. (b) CPU map access latency via PCIe vs GPU-side operations.

Tool	LOC	Domain	gpubpf	NVBit
kernelretsnoop	153	Device	8%	85%
threadhist	89	Device	3%	87%
launchlate	347	Host+Device	14%	93%

Table 1. gpubpf device-side observability tools and overhead comparison with NVBit.

Host Runtime Overhead. We quantify the overhead of the gpubpf driver runtime independent of policy logic using GEMM from PolyBench [18] and HotSpot [6] (1.1× oversubscription, 35 GB working set) on RTX 5090. With hooks enabled but no policy attached, the overhead is less than 0.2%.

Device-side Overhead. We evaluate device-side observability tools built on gpubpf (Table 1) on NVIDIA P40 GPU using llama.cpp prefill (Llama 1B, sequence-mixed). Tools include `kernelretsnoop` (per-block finish timestamps), `threadhist` (load imbalance detection), and `launchlate` (GPU kernel launch latency). Overhead is measured as token/s degradation during prefill. gpubpf achieves 3–14% overhead versus NVBit's 85–93% [44], due to the warp-uniform execution model. Attaching to a running process takes 273 ms on average from `bpftime trace` startup to first GPU kernel detection, without application restart.

Device-side Runtime Optimization. We evaluate device-side gpubpf mechanisms using a vector-add CUDA GPU kernel (32 elements, 10K iterations) from cuda-sample [33] on RTX 5090. Figure 15(a) compares gpubpf's SIMT-aware warp-level execution against eGPU [49] that injects eBPF bytecode without SIMT-specific verification and optimizations. The baseline incurs overhead due to per-thread execution and uncoalesced memory access, while gpubpf's warp-uniform execution and coalesced map access reduce overhead by 60–80%. Figure 15(b) shows CPU map access via PCIe is 6000× slower than GPU-side operations, motivating gpubpf's hierarchical map design that keeps hot state on GPU. We do not compare policy-level performance against the prior workshop version or Neutrino because they are limited to read-only observability (§2).

6 Related Work

`sched_ext` [29], `cache_ext` [54], and XDP extend Linux subsystems via synchronous eBPF callbacks at fixed decision points; gpubpf generalizes this to asynchronous, cross-device effects on user-defined events (§3.2). Driver-level GPU resource managers [11, 14, 23, 24, 47, 51] require unsafe OS kernel driver modifications and service interruptions to change policy. User-space frameworks [7, 16, 31, 38] offer programmability but lack cross-tenant visibility and privileged hardware control (§2.3). gpubpf offers runtime-programmable policies without either limitation. NVBit [44] and Neutrino [21] inject instrumentation into GPU binaries for device-side visibility, but lack safety guarantees and target offline profiling rather than online policy enforcement. A prior workshop paper [49] extends eBPF to GPU instrumentation but remains limited to read-only observability with high overhead (§5) due to per-thread scalar execution on SIMT hardware; gpubpf introduces SIMT-aware verification and warp-level execution for safe, low-overhead policy enforcement.

7 Conclusion

We presented gpubpf, a policy runtime that models each GPU resource as an asynchronous state machine whose state transitions are directed by verified eBPF programs across the host-device boundary. Across four workloads, gpubpf improves throughput by up to 1.76× over application-tuned baselines and reduces tail latency by up to 2×, demonstrating that OS kernel extensibility extends effectively to GPU management.

Acknowledgments

This work used Claude as a generative AI tool for code editing and language improvements.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.

[2] AMD ROCm Documentation 2024. *GPU memory*. AMD ROCm Documentation. https://rocm.docs.amd.com/en/docs-6.2.0/conceptual/gpu-memory.html https://rocm.docs.amd.com/en/docs-6.2.0/conceptual/gpu-memory.html.

[3] Pratheek B, Guilherme Cox, Jan Vesely, and Arkaprava Basu. 2024. SUV: Static Analysis Guided Unified Virtual Memory. In *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO '24)*. IEEE Press, 293–308. doi:10.1109/MICRO61859.2024.00030

[4] Maximilian Bachl, Joachim Fabini, and Tanja Zseby. 2021. A flow-based IDS using Machine Learning in eBPF. *arXiv preprint arXiv:2102.09980* (2021).

[5] Jiashen Cao, Rathijit Sen, Matteo Interlandi, Joy Arulraj, and Hyesoon Kim. 2023. Gpu database systems characterization and optimization. *Proceedings of the VLDB Endowment* 17, 3 (2023), 441–454.

[6] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W Sheaffer, Sang-Ha Lee, and Kevin Skadron. 2009. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE international symposium on workload characterization (IISWC)*. Ieee, 44–54.

[7] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. 2025. KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 1014–1029.

[8] Audrey Cheng, Shu Liu, Melissa Pan, and Ion Stoica. 2025. Barbarians at The Gate: How AI is Upending Systems Research. ACM SIGOPS Blog. https://www.sigops.org/2025/barbarians-at-the-gate-how-ai-is-upending-systems-research/

[9] Yihua Cheng, Yuhan Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. 2025. Lmcache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665* (2025).

[10] LLVM Community. 2025. RFC: SPIR-V IR as a vendor agnostic GPU representation. LLVM Discourse. https://discourse.llvm.org/t/rfc-spirv-ir-as-a-vendor-agnostic-gpu-representation/85115.

[11] Patrick H Coppock, Brian Zhang, Eliot H Solomon, Vasilis Kypriotis, Leon Yang, Bikash Sharma, Dan Schatzberg, Todd C Mowry, and Dimitrios Skarlatos. 2025. LithOS: An operating system for efficient machine learning on GPUs. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 1–17.

[12] Yuran Ding, Xinwei Chen, Xiaofan Zhang, and Zongwei Zhou. 2025. ASAP: an Agentic Solution to Auto-optimize Performance of Large-Scale LLM Training. In *ML for Systems Workshop at NeurIPS*.

[13] Rohit Dwivedula, Divyanshu Saxena, Aditya Akella, Swarat Chaudhuri, and Daehyeok Kim. 2025. Man-Made Heuristics Are Dead. Long Live Code Generators!. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks (HotNets)*. https://arxiv.org/abs/2510.08803

[14] Ruwen Fan, Tingxu Ren, Minhui Xie, Shiwei Gao, Jiwu Shu, and Youyou Lu. 2025. {GPREEMPT}:{GPU} Preemptive Scheduling Made General and Efficient. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. 263–272.

[15] Debashis Ganguly, Ziyu Zhang, Jun Yang, and Rami Melhem. 2019. Interplay between Hardware Prefetcher and Page Eviction Policy in CPU-GPU Unified Virtual Memory. In *ISCA*.

[16] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. 2025. Pie: A Programmable Serving System for Emerging LLM Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 415–430.

[17] Google DeepMind. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. Google DeepMind Technical Report. https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/

[18] Scott Grauer-Gray, Lifan Xu, Robert Searles, Sudhee Ayalasomayajula, and John Cavazos. 2012. Auto-tuning a high-level language targeted to GPU codes. In *2012 innovative parallel computing (InPar)*. Ieee, 1–10.

[19] Yongbin Gu, Wenxuan Wu, Yunfan Li, and Lizhong Chen. 2020. Uvmbench: A comprehensive benchmark suite for researching unified virtual memory in gpus. *arXiv preprint arXiv:2007.09822* (2020).

[20] Pouya Hamadanian, Pantea Karimi, Arash Nash-Esfahany, Kimia Noorbakhsh, Joseph Chandler, Ali Parandeh, Mohammad Alizadeh, and Hari Balakrishnan. 2025. Glia: A Human-Inspired AI for Systems Design and Optimization. ACM SIGOPS Blog. https://www.sigops.org/2025/glia-a-human-inspired-ai-for-systems-design-and-optimization/

[21] Songlin Huang and Chenshu Wu. 2025. Neutrino: Fine-grained {GPU} Kernel Profiling via Programmable Probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 331–355.

[22] Nathan Jones, Tyler Allen, and Rong Ge. 2025. HELM: Characterizing Unified Memory Accesses to Improve GPU Performance under Memory Oversubscription. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. Association for Computing Machinery, New York, NY, USA, 490–504. doi:10.1145/3712285.3759812

[23] Shinpei Kato, Karthik Lakshmanan, Ragunathan Rajkumar, and Yutaka Ishikawa. 2011. {TimeGraph}:{GPU} Scheduling for {Real-Time}{Multi-Tasking} Environments. In *2011 USENIX Annual Technical Conference (USENIX ATC 11)*.

[24] Shinpei Kato, Michael McThrow, Carlos Maltzahn, and Scott Brandt. 2012. Gdev:{First-Class}{GPU} Resource Management in the Operating System. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*. 401–412.

[25] Jens Kehne, Jonathan Metter, and Frank Bellosa. 2019. A Framework for Memory Oversubscription Management in Graphics Processing Units.

[26] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[27] Mao Lin, Yuan Feng, Guilherme Cox, and Hyeran Jeon. 2025. Forest: Access-aware GPU UVM Management. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 137–152. doi:10.1145/3695053.3731047

[28] Yinnian Lin, Lei Zou, and Xunbin Su. 2024. Towards Sufficient GPU-accelerated Dynamic Graph Management: Survey and Experiment. *Proceedings of the VLDB Endowment* 18, 3 (2024), 599–612.

[29] Meta and Linux Kernel Community. 2023. sched_ext: Extensible Scheduler Class. https://github.com/sched-ext/scx.

[30] Nurlan Nazaraliyev, Elaheh Sadredini, and Nael Abu-Ghazaleh. 2025. DREAM: Device-Driven Efficient Access to Virtual Memory. In *Proceedings of the 39th ACM International Conference on Supercomputing (ICS '25)*. Association for Computing Machinery, New York, NY, USA, 1190–1205. doi:10.1145/3721145.3725748

[31] Kelvin KW Ng, Henri Maxime Demoulin, and Vincent Liu. 2023. Paella: Low-latency model serving with software-defined gpu scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 595–610.

[32] NVIDIA. 2022. NVIDIA Open GPU Kernel Modules. https://github.com/NVIDIA/open-gpu-kernel-modules Accessed: 2025.

[33] NVIDIA. 2025. NVIDIA CUDA Samples. https://github.com/NVIDIA/cuda-samples. Accessed: 2025-12-11.

[34] NVIDIA. 2025. Work Stealing with Cluster Launch Control — CUDA Programming Guide. https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cluster-launch-control.html Accessed: 2026.

[35] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *The VLDB Journal* 33, 5 (2024), 1591–1615.

[36] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 3505–3506.

[37] John Ravi, Tri Nguyen, Huiyang Zhou, and Michela Becchi. 2021. PILOT: a Runtime System to Manage Multi-tenant GPU Unified Memory Footprint. In *2021 IEEE 28th International Conference on High Performance Computing, Data, and Analytics (HiPC)*. IEEE, 442–447.

[38] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. 2025. {XSched}: Preemptive scheduling for diverse {XPUs}. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 671–692.

[39] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.

[40] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[41] The Linux Kernel Documentation 2023. *GPU Shared Virtual Memory (GPU-SVM)*. The Linux Kernel Documentation. https://docs.kernel.org/gpu/drm-mm.html Section "DRM GPU scheduler" and GPU-SVM overview, https://docs.kernel.org/gpu/drm-mm.html.

[42] The Linux Kernel Documentation 2024. *DRM GPU scheduler*. The Linux Kernel Documentation. https://docs.kernel.org/gpu/drm-mm.html https://docs.kernel.org/gpu/drm-mm.html.

[43] The Linux Kernel Documentation 2024. *User Mode Queues*. The Linux Kernel Documentation. https://docs.kernel.org/gpu/amdgpu/userq.html https://docs.kernel.org/gpu/amdgpu/userq.html.

[44] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W Keckler. 2019. Nvbit: A dynamic binary instrumentation framework for nvidia gpus. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 372–383.

[45] Guan Wang, Sijie Cheng, Xianyuan Zhan, Xiangang Li, Sen Song, and Yang Liu. 2023. OpenChat: Advancing Open-source Language Models with Mixed-Quality Data. Describes the ShareGPT dataset of 70k user-shared conversations.

[46] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D Owens. 2016. Gunrock: A high-performance graph processing library on the GPU. In *Proceedings of the 21st ACM SIGPLAN symposium on principles and practice of parallel programming*. 1–12.

[47] Yidi Wang, Cong Liu, Daniel Wong, and Hyoseung Kim. 2024. GCAPS: GPU context-aware preemptive priority-based scheduling for real-time tasks. *arXiv preprint arXiv:2406.05221* (2024).

[48] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. 2018. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 595–610.

[49] Yiwei Yang, Tong Yu, Yusheng Zheng, and Andrew Quinn. 2025. eGPU: Extending eBPF Programmability and Observability to GPUs. In *Proceedings of the 4th Workshop on Heterogeneous Composable and Disaggregated Systems*. 73–79.

[50] Peifeng Yu and Mosharaf Chowdhury. 2020. Fine-grained GPU sharing primitives for deep learning applications. *Proceedings of Machine Learning and Systems* 2 (2020), 98–111.

[51] Shulai Zhang, Ao Xu, Quan Chen, Han Zhao, Weihao Cui, Zhen Wang, Yan Li, Limin Xiao, and Minyi Guo. 2025. Efficient Performance-Aware GPU Sharing with Compatibility and Isolation through Kernel Space Interception. In *USENIX Annual Technical Conference (ATC)*.

[52] Yusheng Zheng, YanPeng Hu, Wei Zhang, and Andi Quinn. 2025. Towards Agentic OS: An LLM Agent Framework for Linux Schedulers. In *ML for Systems Workshop at NeurIPS*.

[53] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, XiaoZheng Lai, Dan Williams, and Andrew Quinn. 2025. Extending Applications Safely and Efficiently. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*.

[54] Tal Zussman, Ioannis Zarkadas, Jeremy Carin, Andrew Cheng, Hubertus Franke, Jonas Pfefferle, and Asaf Cidon. 2025. cache_ext: Customizing the Page Cache with eBPF. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 462–478.